COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/14/2026	**DATE SUBMITTED:**	7/15/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 1.1 **Hands-on Activity 1.1 Basic C++ Programming**

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

For this activity, you have to demonstrate the use of a **function prototype**. The section on class declaration and method definition will be used for the function prototype and the function will be defined in the follow method definition section after the main function.

A function prototype in c++ is a declaration of the name, parameters and return type of the function before its definition. Write a C++ code the satisfies the following:

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

Note:

- The driver program must call each function.
- The definitions must be after the main function

COMPUTER ENGINEERING

QUEZON CITY

CODE:

HandOnActivty > ILOA.cpp > ...

```cpp
#include<iostream>

//Create a function that will take two numbers and display the sum
int sumOfTwoNum(int fNum, int sNum);

//Create a function that will return whether variable A is greater than variable B
int isGreater(int varA, int varB);

//Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success
bool logicOps(bool xVal, bool yVal);

int main(){

    std::cout << "The sum of two numbers is: "<< sumOfTwoNum(6,7) << std::endl;

    if(isGreater(6,9) == 1){
        std::cout << "\nvariable A is greater than variable B" << std::endl;
    }else{
        std::cout << "\nvariable A is not greater than variable B" << std::endl;
    }

    if(logicOps(true,true)){
        std::cout << "\nSuccess!" << std::endl;
    }

    return 0;
}

int sumOfTwoNum(int fNum, int sNum){
    int sum = fNum + sNum;
    return sum;
}

int isGreater(int varA, int varB){
    int answer = varA > varB;
    if(answer == 1){
        return 1;
    }else{
        return 0;
    }
}

bool logicOps(bool xVal, bool yVal){
    std::cout << "\nLogical Operations:" << std::endl;
    std::cout << "xVal AND yVal = " << (xVal && yVal) << std::endl;
    std::cout << "xVal OR yVal = " << (xVal || yVal) << std::endl;
    std::cout << "NOT xVal = " << (!xVal) << std::endl;
    std::cout << "NOT yVal = " << (!yVal) << std::endl;
    std::cout << "xVal XOR yVal = " << (xVal != yVal) << std::endl;

    return true;
}
```

OUTPUT:

```
The sum of two numbers is: 13

variable A is not greater than variable B

Logical Operations:
xVal AND yVal = 1
xVal OR yVal = 1
NOT xVal = 0
NOT yVal = 0
xVal XOR yVal = 0

Success!
PS C:\Users\Justine\Desktop\CPE21S4- Braganza>
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements
In this section, the initial implementation for a class triangle will be implemented. The step-by-step procedure is shown below:

Step 1. Include the necessary header files. For this one, we only need #include <iostream>
Step 2. Create the triangle class. Assign it with private variables: totalAngle, angleA, angleB, and angleC.

```
class Triangle{
private:
        double totalAngle, angleA, angleB, angleC;
```

Step 3. We then create public methods. The constructor must allow for creation of the object with 3 initial angles to be stored in our previously defined variables angleA, angleB and angleC. Another method has to be made if the user wants to change the initial values, this will also accept 3 arguments to change the values in angleA, angleB and angleC. Lastly, a function to validate whether the given values make our shape an actual triangle.

```
public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
};
```

Step 4. Define the methods.

```
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    7
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A+B+C;
}
const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Step 5. Create the driver code.

```
int main(){
    //driver code
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
```

```
        std::cout << "The shape is a valid triangle.\n";
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }
    return 0;
    }
```

Include the output of running this code in section 6. Note your observations and comments

CODE:

HandOnActivity > ILOB.cpp > ...

```cpp
#include <iostream>

class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;

public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
};

int main() {
    // driver code
    Triangle set1(40, 30, 110);
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }
    return 0;
}

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}

```

OUTPUT SECTION:

Sections	Answer
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	`class Triangle{ private: double totalAngle, angleA, angleB, angleC; public: Triangle(double A, double B, double C); void setAngles(double A, double B, double C); const bool validateTriangle(); };`

Class Declaration and Method Definition Section

```
class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;

public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle();
};
```

```
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Main Function

```
int main() {
    // driver code
    Triangle set1(40, 30, 110);
    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }
    return 0;
}
```

Method Definition

```
Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Table 1-1. C++ Structure Code for Answer

```
The shape is a valid triangle.
PS C:\Users\TIPQC\Desktop\CPE21S4- Braganza>
```

The program successfully checks whether the given angles form a triangle by calculating their total and using the validateTriangle() function. For the input values 40°, 30°, and 110°, the sum is 180°, so the program correctly outputs that the shape is a valid triangle. The code demonstrates the use of a constructor to initialize the object's values and a member function to validate the triangle.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

1. Create a C++ program to swap the two numbers in different variables.

CODE:

HandOnActivty > SupAct.cpp > main()

```cpp
#include <iostream>
#include <limits>

int main() {

    int swapNum1, swapNum2, temp;

    std::cout << "\n| Number Swapper ˚₊‧꒰ა |\n" << std::endl;
    std::cout << "Enter the first number: ";

    while (!(std::cin >> swapNum1)) {
        std::cout << "Invalid input! Please enter a valid number: ";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

        std::cout << "Enter the second number: ";

    while (!(std::cin >> swapNum2)) {
        std::cout << "Invalid input! Please enter a valid number: ";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    std::cout << "\nBefore swapping: " << std::endl;
    std::cout << "First number: " << swapNum1 << std::endl;
    std::cout << "Second number: " << swapNum2 << std::endl;

    // Swapping the numbers
    temp = swapNum1;
    swapNum1 = swapNum2;
    swapNum2 = temp;

    std::cout << "\nAfter swapping: " << std::endl;
    std::cout << "First number: " << swapNum1 << std::endl;
    std::cout << "Second number: " << swapNum2 << std::endl;

    return 0;
}
```

OUTPUT:

```
| Number Swapper ˚₊‧꒰ა ❀ ꒱ |

Enter the first number: No
Invalid input! Please enter a valid number: 21
Enter the second number: Nah
Invalid input! Please enter a valid number: 67

Before swapping:
First number: 21
Second number: 67

After swapping:
First number: 67
Second number: 21
PS C:\Users\Justine\Desktop\CPE21S4- Braganza>
```

Analysis:
This program prompts the user for two inputs, prints out the original values of both, swaps them through the use of a temporary variable, and prints out the swapped values. It utilizes while loops and input validation to ensure that valid input is provided. The biggest challenge involved in developing this program was incorporating the error handling into the program, but after learning the proper use of cin.clear(), cin.ignore(), and looping, the program became easy to write.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

CODE:

HandOnActivity > SupAct2.cpp > main()

```cpp
#include <iostream>
#include <limits>

float KelToFah(float kelvin);
float FahToKel(float fahrenheit);

int main() {

    int choice;
    float temperature;
    int exitProgram = 0;

    while(exitProgram == 0) {
        std::cout << "\n|  ˚·❄ Temperature Converter ˚·❄  |\n" << std::endl;
        std::cout << "Choose an option:\n";
        std::cout << "1. Convert Kelvin to Fahrenheit\n";
        std::cout << "2. Convert Fahrenheit to Kelvin\n";
        std::cout << "3. Exit\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                std::cout << "\nEnter temperature in Kelvin: ";
                while (!(std::cin >> temperature)) {
                    std::cout << "Invalid input! \n\nPlease enter a valid number: ";
                    std::cin.clear();
                    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
                std::cout << temperature << " K = " << KelToFah(temperature) << " °F" << std::endl;
                break;
            case 2:
                std::cout << "\nEnter temperature in Fahrenheit: ";
                while (!(std::cin >> temperature)) {
                    std::cout << "Invalid input! \n\nPlease enter a valid number: ";
                    std::cin.clear();
                    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                }
                std::cout << temperature << " °F = " << FahToKel(temperature) << " K" << std::endl;
                break;
            case 3:
                exitProgram = 1;
                std::cout << "\nExiting the program. Goodbye!" << std::endl;
                break;
            default:
                std::cout << "Invalid choice! \n\nPlease try again." << std::endl;
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            }

    }
        return 0;
    }

    float KelToFah(float kelvin) {
        return (kelvin - 273.15) * 9/5 + 32;
    }

    float FahToKel(float fahrenheit) {
        return (fahrenheit - 32) * 5/9 + 273.15;
    }
```

OUTPUT:

```
| ˚₊‧꒰❀ Temperature Converter ˚₊‧꒰❀ |

Choose an option:
1. Convert Kelvin to Fahrenheit
2. Convert Fahrenheit to Kelvin
3. Exit
Enter your choice: Why
Invalid choice!

Please try again.

| ˚₊‧꒰❀ Temperature Converter ˚₊‧꒰❀ |

Choose an option:
1. Convert Kelvin to Fahrenheit
2. Convert Fahrenheit to Kelvin
3. Exit
Enter your choice: 1

Enter temperature in Kelvin: Nope
Invalid input!

Please enter a valid number: 100
100 K = -279.67 °F
```

```
| ˚₊‧꒰❀ Temperature Converter ˚₊‧꒰❀ |

Choose an option:
1. Convert Kelvin to Fahrenheit
2. Convert Fahrenheit to Kelvin
3. Exit
Enter your choice: 2

Enter temperature in Fahrenheit: 67
67 °F = 292.594 K

| ˚₊‧꒰❀ Temperature Converter ˚₊‧꒰❀ |

Choose an option:
1. Convert Kelvin to Fahrenheit
2. Convert Fahrenheit to Kelvin
3. Exit
Enter your choice: 3

Exiting the program. Goodbye!
PS C:\Users\Justine\Desktop\CPE21S4- Braganza>
```

Analysis:

This program will convert temperatures from Kelvin to Fahrenheit and vice versa with two different functions. It is going to show a menu which enables users to select their preferred conversion or exit the program. Furthermore, the program makes use of a 'while' loop to continuously display the menu until the user decides to exit. In addition, the program has been designed to carry out validation checks to ensure that the inputs provided by the users are only numbers. The hardest part or maybe where I struggled the most was the function definition.

3. Create a C++ program that has a function that will calculate the distance between two points.

CODE:

HandOnActivty > SupAct3.cpp > main()

```cpp
#include <iostream>
#include <cmath>
#include <limits>

double calculateDistance(double x1, double y1, double x2, double y2);

int main() {
    double x1, y1, x2, y2;

    std::cout << "\n| ˚₊‧꒰❀ Distance Calculator ˚₊‧꒰❀ |\n";

    std::cout << "\nEnter the coordinates of the first point (x1 y1): \n";
    while (!(std::cin >> x1 >> y1)) {
        std::cout << "Invalid input!\n\nPlease enter valid coordinates: \n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    std::cout << "\nEnter the coordinates of the second point (x2 y2): \n";
    while (!(std::cin >> x2 >> y2)) {
        std::cout << "Invalid input!\n\nPlease enter valid coordinates: \n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    double distance = calculateDistance(x1, y1, x2, y2);

    std::cout << "\nThe distance between the two points is: " << "\n" << distance << std::endl;

    return 0;
}

double calculateDistance(double x1, double y1, double x2, double y2) {
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}
```

OUTPUT:

```
| ˚₊‧꒰❀ Distance Calculator ˚₊‧꒰❀ |

Enter the coordinates of the first point (x1 y1):
NO
Invalid input!

Please enter valid coordinates:
100
67

Enter the coordinates of the second point (x2 y2):
44
81

The distance between the two points is:
57.7235
PS C:\Users\Justine\Desktop\CPE21S4- Braganza>
```

Analysis
The purpose of this program is to calculate the distance between two points in a Cartesian coordinate system using the distance formula. This program requests the user to input the coordinates of two points, verifies that the input is valid (i.e., that only valid numbers are entered), and then uses a function to calculate the distance through the use of sqrt and pow functions. I had trouble mainly with the calculations because I did not use <cmath> and performed the power and square root functions manually, but after finding an easier way, my code was finished.

4. Modify the code given in ILO B and add the following functions:
    a. A function to compute for the area of a triangle
    b. A function to compute for the perimeter of a triangle
    c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.

CODE:

HandOnActivty > SupAct4ILOB.cpp > typeTria()

```cpp
#include <iostream>

class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;
    double base, height, area;

public:
    Triangle(double A, double B, double C, double bs, double ht);
    void setAngles(double A, double B, double C);
    void setArea(double bs, double ht);
    double displayArea();
    double perimeteraTria(double side1, double side2, double side3);
    void typeTria();
    const bool validateTriangle();
};

int main() {
    // Driver code
    Triangle set1(60, 60, 60, 10, 5);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
        std::cout << "Area: " << set1.displayArea() << "\n";
        std::cout << "Perimeter: " << set1.perimeteraTria(10, 10, 10) << "\n";
        set1.typeTria();
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C, double bs, double ht) {
    angleA = A;
    angleB = B;
    angleC = C;
    base = bs;
    height = ht;
    totalAngle = A + B + C;
    area = 0.5 * base * height;
}

```

```cpp
void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setArea(double bs, double ht) {
    base = bs;
    height = ht;
    area = 0.5 * base * height;
}

double Triangle::displayArea() {
    return area;
}

double Triangle::perimeteraTria(double side1, double side2, double side3) {
    return side1 + side2 + side3;
}

void Triangle::typeTria() {
    if (angleA == 90 || angleB == 90 || angleC == 90) {
        std::cout << "A Right Triangle" << std::endl;
    } else if (angleA > 90 || angleB > 90 || angleC > 90) {
        std::cout << "An Obtuse Triangle" << std::endl;
    } else {
        std::cout << "An Acute Triangle" << std::endl;
    }
}

const bool Triangle::validateTriangle() {
    return (totalAngle == 180);
}
```

OUTPUT:

```
The shape is a valid triangle.
Area: 25
Perimeter: 30
An Acute Triangle
PS C:\Users\Justine\Desktop\CPE21S4- Braganza>
```

Analysis:

This program implements OOP through the creation of the Triangle class with the properties of a triangle, namely three angles, base, height, total angle and area. In the case of setting an object set1, the constructor initializes all these values as well as calculates total angle and the area based on the formula (0.5 x base x height). validateTriangle() function verifies whether the sum of three angles is 180 degrees, which shows if the triangle is

valid or not. If it is a valid triangle, then the area is displayed through displayArea(), while perimeter is calculated by summing up three side lengths with perimeteraTria(). Furthermore, the kind of triangle (acute, right, or obtuse) is found using typeTria() function. Every process is implemented using separate functions, so that the code can be easily understood, reused and modified. The hardest thing was to understand how constructors initialize objects and what is the role of member functions. I struggled the most here, with the functions, computations and just coding in general, I still lack a lot in this department and this was heavily AI guided. I understand what it does, but if I were to make this from scratch I don't think I would be able to finish or complete anything without any help from guides.

CONCLUSION:

This hands-on activity was a good refresher for me because I lack a lot of C++ coding knowledge. I forgot most of the things I've encountered in first year, but the good thing is I am familiar with most of the tasks that were given to me in this activity. The step-by-step procedure also helped me recall the proper syntax and structure of C++ programs, making it easier to understand how each part of the code works together. I've learned more about classes and how they are used to make user-defined data types that serve as a blueprint or template for creating objects, as well as how constructors and member functions work together to initialize and manipulate an object's data. I also learned how encapsulation helps organize data and functions into a single class. The supplementary activities gave me more opportunities to apply these concepts by creating different programs and analyzing how they function. How it bundles related data variables and functions (called methods) into a single, cohesive unit. I've also had a review on how to define functions in C++ because it is completely different from Python, so I had to recall how to do them during this activity.

I will be honest though, the triangle task was quite difficult for me to understand because of how it's structured, and I still don't have that much knowledge to be able to handle OOP 2 yet. It took me some time to understand how the constructor, validation function, and other member functions interacted with one another, but after studying the code more carefully, I gradually understood their purpose. I hope in the future I will be able to understand and appreciate C++ coding. The farther I go into this course, the more I realize I am not the brightest, but I will learn as much as I can despite the difficulties. Overall, I think I completed the activity fairly well, although there are still many concepts that I need to improve on, especially object-oriented programming, problem-solving, and writing more efficient C++ code. I believe that with more practice and consistent coding exercises, I will become more confident in using C++ and applying OOP concepts in future activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, code assistance and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

I used AI to help me recall how to make functions and how to define them. I asked for help with certain concepts that seemed difficult for me to understand so I would be able to code by myself. I also used AI to rephrase some of my code analysis to make it more comprehensive when I read back to it.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.